

Design Rationale

1. Dictionaries for Books

Dictionaries were chosen to store book information because each book can be uniquely identified using an **ISBN (International Standard Book Number)**.

In Python, dictionaries are **key-value** pairs, making them ideal for representing a collection where each key (ISBN) maps directly to its associated data (book details).

Each ISBN acts as a **unique key**, ensuring that no two books share the same identifier. This provides:

- **Fast lookups:** Retrieving or updating a book's details is extremely efficient (average time complexity **$O(1)$**).
- **Clarity in structure:** Each book entry contains related attributes (title, author, genre, total copies, available copies) stored together in a single nested dictionary.
- **Ease of update and deletion:** Since each book is referenced by its ISBN, editing or removing a book record requires minimal code and no iteration over the entire dataset.

2. Lists for Members

Members are stored in a **list of dictionaries**, as the member data does not require a unique global key like ISBN.

Instead, a list allows multiple member records to coexist and be iterated through easily.

Each list item is a dictionary representing one member's profile, containing keys such as **member_id**, **name**, **email**, and **borrowed_books**.

Advantages of using a list:

- **Flexible storage:** Lists allow appending new members dynamically as the library grows.
- **Ease of traversal:** Searching or updating a member's details can be done using simple iteration.
- **Simplicity:** Perfect for small-to-medium datasets where simplicity is more important than indexing efficiency.

3. Tuples for Genres

Genres are stored as a **tuple** because the set of allowed genres is predefined and should not change while the system runs.

A tuple is **immutable**, meaning once created, its contents cannot be altered — this makes it ideal for constants and reference data.

Advantages:

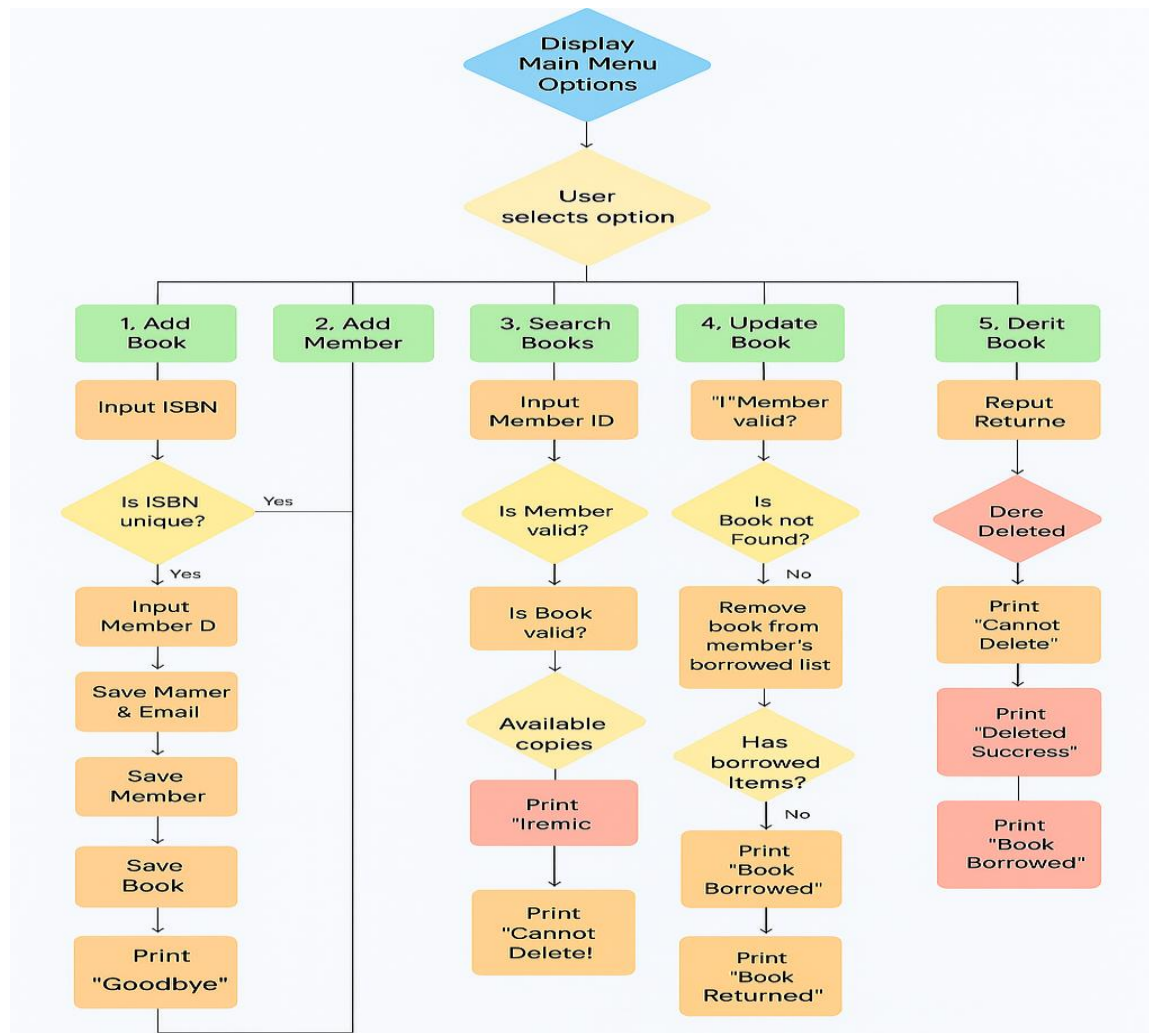
- **Data integrity:** Prevents accidental modification of genre names.
- **Validation support:** Used to check if a book's genre is valid before adding it to the system.
- **Memory efficiency:** Tuples use less memory than lists and provide faster read access.

4. Functions for Modularity and Code Reuse

Instead of writing one large block of procedural code, the program is divided into **separate, reusable functions** — each responsible for a specific operation (add, update, delete, borrow, return).

This modular design offers several benefits:

- **Readability:** Each function name clearly describes its purpose, making the code easy to follow.
- **Maintainability:** Future updates (e.g., adding validation or database support) can be done within specific functions without affecting others.
- **Reusability:** Functions like `add_book()` or `borrow_book()` can be imported into other Python scripts or reused in larger projects.
- **Testing:** Smaller, isolated functions simplify debugging and allow for targeted unit tests using `assert`.



Above is the flowchart visual description.